

Heuristics and deep reinforcement learning for the inventory problem with an all-or-nothing yield pattern and non-zero leadtimes

Yuting Yan ^{a,b}, Frank (Youhua) Chen ^b, Zhe Fu ^b, Wenjie Bi ^{a,*}

^a Business School, Central South University, Changsha, Hunan 410083, China

^b Department of Decision Analytics and Operations, City University of Hong Kong, 999077, Hong Kong, China

ARTICLE INFO

Keywords:

Inventory
Random yield
Reinforcement learning
Deep learning

ABSTRACT

We consider the single-item, periodic-review inventory system with stochastic demand, an all-or-nothing random yield pattern, and non-zero leadtimes, where the unmet demand is backlogged. The firm's objective is to find an optimal ordering policy to minimize its total expected discounted cost. As the optimal policy remains unknown, we first develop a new heuristic that takes advantage of the all-or-nothing yield pattern and then propose a deep reinforcement learning (DRL) algorithm based on a tailored neural network model. The proposed DRL algorithm combines the results of existing heuristics to improve performance and stabilize the training process. Moreover, the total cost to be computed is convex in the decision variable, which facilitates the search. Extensive numerical experiments show that our new solution methods outperform known heuristics and demonstrate near-optimal performance when leadtimes are relatively short and the minimal cost can be achieved numerically.

1. Introduction

This research is motivated by one of the co-authors' project experiences at a pharmaceutical firm specializing in antibiotic drugs that requires a framework for replenishing procured key ingredients/materials. During its fulfilling process, active ingredients and auxiliary materials (e.g., starch and capsules) mixed or in direct contact with drugs can cause quality risks. These incoming material shipments must be sampled to screen out microbial contamination. Thus, the firm conducts a sample test immediately upon the arrival of each order shipment before production use. The entire shipment will be disposed of or returned if the sample test fails. This yield characteristic is referred to as an all-or-nothing yield pattern. The all-or-nothing type of yield is especially relevant in biomanufacturing due to random shocks that can affect the entire production (Tomlin, 2009; Martagan et al., 2016, 2020). For example, during the freeze-drying process, physico-chemical process parameters (i.e., temperature, moisture, PH, etc.) might deviate from their pre-specified control ranges due to bacterial contamination, human errors, or equipment failure. In such cases, the whole shipment needs to be scrapped to comply with regulatory requirements after receiving the shipment.

As a niche player in a high-end market, the pharmaceutical firm used to enjoy a healthy profit margin. However, as new competitors entered the market, the management began to pay more attention to its manufacturing operations and supply chain processes. For example,

production planning and materials procurement was primarily done manually by experienced managers with the aid of spreadsheet tools. The project was completed with a recommendation that the firm adopt a certain amount of safety stock for each commonly used material. The safety stock was calculated based on the concept of the linear inflation rule (LIR), where orders are placed only when inventory positions (the inventory level plus in-transit pipeline orders assumed of "expected yield") are below certain target inventory levels (see Section 4.2 below).

With the above motivational examples, the current paper considers the single-item, periodic-review inventory system with stochastic demand, an all-or-nothing random yield pattern, and non-zero leadtimes. The objective is to find an efficient inventory replenishment policy that tries to minimize the expected discounted cost, including purchasing, inventory-holding, and backlogging costs over the planning horizon. As well understood, the structure of an optimal inventory control policy for such a problem is likely to be complex due to the intrinsic challenges of problems with an all-or-nothing random yield model and leadtimes. On the one hand, this inventory system must cope with uncertainties from both the demand and production sides, facing the possibility that some order shipments in transit may become non-usable. On the other hand, the dynamic programming (DP) model of this problem also suffers from the curse of dimensionality, which

* Correspondence to: Central South University, Room 379, Jiangwan Building, China.

E-mail addresses: yutingyan2-c@my.cityu.edu.hk (Y. Yan), youhchen@cityu.edu.hk (F. Chen), zhufu4-c@my.cityu.edu.hk (Z. Fu), bwj@csu.edu.cn (W. Bi).

means that the state space becomes exponentially large as the length of leadtime increases, and obtaining an optimal solution to such a DP problem becomes computationally intractable. Without knowledge of the optimal solution's structure, exact numerical methods, such as value and policy improvement iteration algorithms (Puterman, 2014), become impractical.

Although several heuristics have been proposed and tested under a general form of yield randomness, only a few are known for the problem with an all-or-nothing yield pattern. Past studies (Inderfurth, 2009; Inderfurth and Vogelgesang, 2013; Inderfurth and Kiesmüller, 2015) have developed a linear inflation rule (LIR), where orders are placed only when inventory positions are below certain target inventory levels. If the gap between the target level and the current inventory position is positive, then the order quantity is set equal to the difference multiplied by the reciprocal of the mean yield rate. Although this rule considers demand and yield uncertainty when calculating target stock levels, it is too simple to consider yield variability when determining order quantities. Instead of using the inverse of the mean yield rate as a yield inflation factor, Chen et al. (2018) propose a weighted news-vendor heuristic (WNH) that explicitly considers all possible realizations of pipeline orders. Roughly speaking, the WNH approximation is a random-yield version of the myopic solution to the deterministic-yield setting in which the order quantity is determined by the gap between the target inventory level (same value as in the LIR) and the beginning inventory levels in the $(L + 1)$ th period, where L is the leadtime. This heuristic is derived based on a sample-path approach: for each realization of the pipeline orders, there is a corresponding gap, and it then takes a weighted average of these gaps, where the weights are the corresponding probabilities of the yield sample paths of pipeline orders (see Section 4.2 for more details). Clearly, the approximation takes advantage of the all-or-nothing yield pattern. This resulting policy performs well in their numerical simulations. However, this method is still a myopic solution and fails to account for the functional dependence of these weights on the current inventory level and pipeline orders. Therefore, the current paper is motivated to improve upon these known heuristics.

The primary objective of this paper is to find an algorithm that can improve upon known heuristics. For this purpose, the first aim of this paper is to develop a new heuristic that takes advantage of the all-or-nothing yield pattern. We draw on the structure of WNH and propose a new heuristic that adopts the gradient descent algorithm to search the optimal state-independent weights with the objective of minimizing the total expected discounted cost over the planning horizon (see Section 4.3 for more details). We prove that the total cost to be computed is convex in the weight vector, which facilitates the search.

The second aim of this paper is to explore the potential of applying DRL techniques to learn an optimal ordering policy. Reinforcement learning (RL) can efficiently solve sequential decision-making problems, while deep neural network (DNN) can effectively circumvent the curse of dimensionality issue (Boute et al., 2022). By reformulating a problem as a Markov decision process (MDP) with state transitions, a DNN (or even a neural network, NN) can approximate action value or policy functions. Combining both RL and DNN gives rise to DRL. Then, the DRL agent can learn excellent strategies by taking actions in a trial-and-error way, interacting with the environment, and generating reward or punishment signals. In recent years, DRL has achieved impressive success in terms of finding winning strategies in games and beyond. Notable examples include deep model-free Q-learning (DQN) for classic Atari 2600 games (Mnih et al., 2015) and the Monte Carlo simulation policy combined with the search algorithm AlphaGo, which resulted in defeating top human experts in Go games (Silver et al., 2016). There have also been numerous exploratory studies from different aspects of the field of inventory management in recent years. For example, Gijbrecchts et al. (2022) demonstrate that the Asynchronous Advantage Actor Critic (A3C) algorithm can effectively

solve three classic inventory problems: dual sourcing, lost sales, and multi-echelon inventory management. (Oroojlooyjadid et al., 2022) propose the shaped-reward DQN (SRDQN) algorithm, which learns a near-optimal policy in the classic Beer game problem. (De Moor et al., 2022) leverage domain knowledge from heuristics to improve the performance of deep Q-network algorithm in a perishable inventory problem. Kaynov et al. (2024) apply a DRL solution based on a proposed random rationing policy for one-warehouse multi-retailer inventory management. Wang and Minner (2024) apply the proximal policy optimization (PPO) algorithm to address a demand fulfillment problem in online retailing with different distribution centers, which has also achieved promising outcome. Given the DRL algorithms' excellent performance, this paper tries to apply DRL techniques to solve the inventory system with an all-or-nothing random yield pattern.

To achieve the second aim, this paper develops a customized DNN model that takes advantage of the particular structure of the yield pattern to output state-dependent weights. A DRL algorithm then implements this DNN model as a policy function approximator to learn an ordering strategy to minimize the expected total discounted cost. This well-structured function approximator allows faster convergence when deploying the DRL algorithm. Furthermore, from the implementation perspective, the proposed DRL algorithm integrates the results from a known heuristic to improve DRL performance and stabilize the training process. Specifically, the known heuristic is fed into the new DRL algorithm as an initial solution. Through a reward-shaping mechanism, the DRL algorithm learns to improve the results from the heuristic.

Although the data used in our experiments is generated by sampling from chosen distributions for demand and yield rate over time, the proposed two methods do not make any assumption on either distribution. In other words, the underlying methods are data-driven, which sets our paper apart from the existing literature on random yield inventory problems. So far, little is known about the optimal solution for this periodic review inventory policy with random yield and non-zero lead-times. Through a relatively extensive numerical study, the proposed two methods have been evaluated against two known benchmark policies, and the results show these new methods outperform benchmarks at least with a 2% cost gap. When the leadtime is small, where exact DP numerical methods can be used to solve the Bellman equation, the minimal cost is achievable; in this case, we can obtain the (globally) minimal total discounted cost. Our methods have demonstrated near-optimal performance compared to the minimal cost achieved by DP and better performance to other known policies.

The rest of the paper is organized as follows. Section 2 reviews the closely related literature. Section 3 describes the inventory model. Section 4 proposes a new heuristic and a DRL algorithm as solution methods. Section 5 reports the results of an intensive numerical study. Section 6 concludes the paper with a summary.

2. Literature review

Inventory models with random yield and stochastic demand have drawn attention from many researchers in the operations management community. We mainly focus our review on studies that investigate random yield models and relevant approximation methods. For ease of exposition, the related literature can be categorized into zero leadtime and non-zero leadtimes cases. For a comprehensive review, we refer to Yano and Lee (1995) for random yield risks and Inderfurth and Vogelgesang (2013) for the approximation methods.

For the case of zero leadtime, Henig and Gerchak (1990) provide a comprehensive analysis of the stochastically proportional yield model in a general periodic review production/inventory model, proving that the optimal inventory replenishment policy involves a time-dependent critical target stock level. According to this policy, orders are placed if the current inventory level does not exceed this target stock threshold in a single supply source. Unlike a base stock rule used under deterministic yield conditions, this policy incorporates a non-linear

relationship between the order quantity and the inventory position. Due to its computational complexity and impracticality, a simple approximation approach is helpful. Several of these approaches, including the newsvendor and NLH-2 heuristic introduced by [Bollapragada and Morton \(1999\)](#) as well as the heuristic developed by [Li et al. \(2008\)](#), aim to approximate the optimal rule using a more straightforward non-linear function. Alternatively, the order quantity for more practical policies ([Huh and Nagarajan, 2010](#)) follows a linear relationship of the deviation between the target stock level and the inventory position. This type of policy is known as LIR. This rule offers a practical and efficient approach for determining order quantities in an inventory system with a random yield model by simplifying the decision-making process and reducing computational effort. However, these methods mainly consider the zero leadtime setting. [Federgruen and Yang \(2011\)](#) generalize the model proposed by [Henig and Gerchak \(1990\)](#) to include multiple suppliers. In particular, they develop an efficient algorithm to find the optimal sourcing strategy from multiple suppliers with random yield and price characteristics. [Li \(2017\)](#) investigate optimal procurement strategies from two suppliers with random yield and all-or-nothing risks. Our research is also related to the system disruption literature, in which random disruptions can be considered as a special case of the stochastically proportional yield model and are equivalent to the all-or-nothing yield pattern: the firm receives either 0% or 100% of the order ([Wang et al., 2010](#); [Tang et al., 2014](#)). Random yield models have many applications in different aspects, including assembly system ([DeCroix, 2013](#)), different payment schemes ([Kouvelis et al., 2018](#)), and supply chain finance ([Yuan et al., 2021](#)), but the focus is quite different from the main body of this work.

In the extensive literature on random yield models, a few published studies report results with specific heuristics for problems with positive leadtimes. For example, [Inderfurth \(2009\)](#) extend the parameter determination approach of [Bollapragada and Morton \(1999\)](#) and develop LIR for non-zero leadtimes cases. [Inderfurth and Vogelgesang \(2013\)](#) further consider a time-dependent critical target stock policy with LIR under stochastic demand and different types of yield randomness settings. They also present safety stock formulas and investigate the performance of the proposed safety stock variants in a numerical study. Based on the study of [Inderfurth and Vogelgesang \(2013\)](#), [Inderfurth and Kiesmüller \(2015\)](#) focus on the class of stationary linear inflation policies and investigate a periodic-review inventory system with binomial and stochastically proportional two different yield models. Compared with a general form of yield randomness, [Chen et al. \(2018\)](#) consider a special case — the all-or-nothing production yield pattern with non-zero leadtimes. They take advantage of the unique form of yield randomness, explicitly consider the uncertainty of the yield in determining order quantities, and develop the WNH policy. [Bu et al., \(2020\)](#) investigate constant-order policies for lost-sales inventory models with non-zero leadtimes and random yield, thereby proving that the best policy is asymptotically optimal with large leadtimes. Our study contributes to this stream of literature by proposing a new DRL algorithm, which is a general technique for sequential decision-making problems that shows better performance compared with LIR and WNH policies in the following numerical simulation section.

3. Model formulation

We consider a single-item, periodic-review inventory system with stochastic demand, an all-or-nothing random yield pattern and non-zero leadtimes in a finite horizon. After each order is received, the firm knows the yield result. In our earlier example, upon receiving the order, the firm immediately takes a sample test; if the sample test fails, the entire shipment is considered defective and either disposed of or returned. We also assume that random yields are independent across orders.

There are T periods in the planning horizon, and we count the periods forward. The leadtime, denoted as $L \geq 0$, represents the total

time including the supplier's order fulfillment process to the completion of the sample test. Let $Y_t \geq 0$ be the random yield variable in period t , and y_t denote the realization of yield Y_t . We consider the all-or-nothing random yield pattern: $Y_t = 1$ with probability p , and $Y_t = 0$ with probability $1 - p$. At the beginning of each period $t \in \{1, \dots, T\}$, the firm first receives the order q_{t-L} , which was placed before L periods. If $y_t = 1$, then a total of q_{t-L} usable units is added to the existing inventory level. Otherwise, i.e., $y_t = 0$, there will be no additional stock for the current period. The resulting inventory level is denoted as I_t ; a positive inventory level means usable stock on hand, and a negative inventory level means an out-of-stock situation, resulting in backorders. With the information of I_t and the vector of outstanding or pipeline orders in transit $\mathbf{Q}_t = (q_{t-1}, q_{t-2}, \dots, q_{t-L+1})$, the firm then decides on the order quantity q_t . The demand d_t is then observed and fulfilled from on-hand inventory as much as possible, and the unmet demand will be backlogged. We assume that demands over time are independent and identically distributed (IID), which will be relaxed later. When the time moves to $t + 1$, the vector of pipeline orders will be transformed by "moving" the order: $\mathbf{Q}_{t+1} = (q_t, q_{t-1}, \dots, q_{t-L+2})$. The inventory level also updates based on the observed demand d_t in period t and yield realization y_{t+1} in period $t + 1$ as $I_{t+1} = I_t + y_{t+1}q_{t+1-L} - d_t$.

The inventory holding and backordering costs are evaluated based on the ending inventory level in period t . Specifically, at each period, the positive left-over inventory incurs a positive unit holding cost c_h , and the unmet demand is backlogged, incurring a per unit backlogging cost c_b . Given that the firm's current inventory level is I_t , the one-period inventory holding and backlogging cost at time step t can thus be defined as: $H(I_t) = c_h[I_t - d_t]^+ + c_b[d_t - I_t]^+$, where $[x]^+ = \max\{x, 0\}$ is the positive part operator. Let c_o represent the time-homogeneous per unit ordering cost over the planning horizon. The firm only pays the purchase cost to the supplier after receiving the order shipment. At the end of each period t , the purchasing, inventory-holding, and backlogging costs are calculated as follows: $c_t(I_t, q_{t-L}) = c_o q_{t-L} y_t + H(I_t)$. Due to the existence of leadtime, the current order decision q_t will not immediately affect the realized cost c_t ; that is, there would be a lag (or delay) effect. In other words, c_t is affected by the orders q_{t-L} placed before L periods. In each period t , the seller chooses an order quantity q_t in an attempt to minimize the total expected discounted cost over the finite planning horizon: $\min_{q_t \geq 0} \mathbb{E} \left[\sum_{t=1}^T \alpha^{t-1} c_t(I_t, q_{t-L}) \right]$. We assume that no outstanding orders arrive during the first L periods of the planning horizon. The notations used in this paper are listed in [Table 1](#).

4. Methodology

In this section, we propose two new solutions to learn a well-performing ordering policy for the inventory control problem with an all-or-nothing random yield pattern. First, we formalize this inventory problem as an MDP. Then, we introduce two known heuristic policies that serve as benchmarks for our new solutions. Finally, we propose a new heuristic with state-independent weights and a new DRL approach with state-dependent weights motivated by WNH.

4.1. MDP formulation

The inventory problem with an all-or-nothing yield pattern can be formalized as an MDP with state $s_t = (I_t, \mathbf{Q}_t)$ and action q_t . Let $\mathcal{S}$ and $\mathcal{A}$ respectively denote the state space and the action space. The firm's objective is to find a policy that consists of a sequence of functions $\Pi = \{\pi_1, \dots, \pi_T\}$, where π_t maps each state s_t into an action $q_t = \pi_t(s_t)$. To this end, the value function of being in the initial state s_1 following policy Π can be defined as $V_1^\Pi(s_1) = \mathbb{E} \left[\sum_{t=1}^T \alpha^{t-1} c_t(I_t, q_{t-L} | q_{t-L} \sim \pi_{t-L}(s_{t-L})) \right]$, with $0 \leq \alpha \leq 1$ the discount factor. An optimal policy $\Pi^* = \{\pi_1^*, \dots, \pi_T^*\}$ is the one that minimizes the total expected discounted cost, i.e., $V_1^{\Pi^*}(s_1) = \min_{\Pi} V_1^\Pi(s_1)$. The

Table 1
Notation table.

Notation	Meaning
α	The discount factor
L	The total order leadtime
c_o	Time-invariant unit ordering cost
c_h	Time-invariant unit holding cost
c_b	Time-invariant unit backlogging cost
S_L	Target inventory level
I_t	Inventory level in period t after accounting for the shipment with its yield being known
$\mathbf{Q}_t$	Vector of pipeline orders that will complete testing in period $t+1$ or later
D_t	Random demand in period t , with d_t being its realization
Y_t	The random yield variable in period t : $y_t = 1$ (w.p. = p), or 0 (w.p. = $(1-p)$)
s_t	State in period t with state space S
q_t	The order quantity (action) in period t with action space $\mathcal{A}$
$H(I_t)$	One-period inventory holding and backlogging cost for period t
$c_t(I_t, q_{t-L})$	Cost incurred in period t
C_t	The total discounted cumulative cost from the initial period to period t

dynamic programming formulation of this inventory problem is given by

$$V_t(I_t, \mathbf{Q}_t) = \min_{q_t \geq 0} \mathbb{E}[c_o q_t Y_{t+L} + \alpha^L H(I_{t+L}) + \alpha V_{t+1}(I_t + Y_{t+1} q_{t+1-L} - D_t, \mathbf{Q}_{t+1})]$$

$$\triangleq \min_{q_t \geq 0} \{F_t(I_t, \mathbf{Q}_t, q_t)\}, \quad (1)$$

where $I_{t+L} = I_t + \sum_{i=1}^L Y_{t+i} q_{t+i-L} - D_{[t,t+L-1]}$ and $D_{[t,t+L-1]} = \sum_{i=0}^{L-1} D_{t+i}$ with boundary conditions $V_{T+1} = 0$. The terms in (1) include the expected purchase cost in period t (we count the purchasing cost until the order is tested after L periods later); the expected holding and shortage costs in the period $t+L$, which is the earliest period that the order quantity in period t can affect; and the discounted expected cost over the remaining horizon.

For this problem, we can find the optimal order quantity $q_t^* = \pi_t^*(s_t)$ for each period t , which depends on the inventory level I_t and the vector of pipeline orders $\mathbf{Q}_t$ by solving the optimal Bellman equations (Bellman, 1966). However, solving these Bellman equations can quickly become computationally intractable because this problem suffers from the curse of dimensionality that the L -dimensional state space becomes exponentially large as leadtime increases (Powell, 2007). Therefore, it is necessary to find a well-performing solution to the problem.

4.2. Benchmark policies

As previously mentioned, there are two known heuristics for our inventory system with an all-or-nothing yield pattern. The first has been proposed in the literature for a general random yield model. The second further considers an all-or-nothing yield pattern that motivates our method. These two heuristics serve as benchmark policies for our proposed solutions.

Linear inflation rule (LIR). In a problem context characterized by stationary stochastic demand and yield processes, LIR is used to minimize the expected long-run average cost without discounting. LIR has been proven effective for inventory problems with stochastic demand and various types of yield randomness in the multiperiod infinite-horizon case (Inderfurth, 2009; Inderfurth and Vogelgesang, 2013; Inderfurth and Kiesmüller, 2015). According to this rule, the order quantity is determined by

$$q_t^{\text{LIR}} = \beta [S_L - \text{IP}_t]^+, \quad (2)$$

where $\text{IP}_t = I_t + p \sum_{i=1}^{L-1} q_{t-i}$ denotes the inventory position in period t upon I_t being known but before the new order is placed in the period; β is the yield inflation factor: this paper sets $\beta = 1$ because we found that the cost of LIR is lower when $\beta = 1$ than when $\beta = 1/p$; S_L represents the target inventory level, which differs in L . Inderfurth

and Vogelgesang (2013) develop an approximation method to calculate the target inventory level S_L for this linear control rule by using the following formula:

$$S_L = (L+1)\mu_D + k \sqrt{(L+1)\sigma_D^2 + \max\{L, 1\} \frac{\rho_Y^2}{1-\rho_Y^2} (\mu_D^2 + \sigma_D^2)}, \quad (3)$$

where μ_D and σ_D represent the mean and standard deviation of demand D , respectively; ρ_Y is the coefficient of variation of the yield factor Y : under the all-or-nothing yield pattern, $\rho_Y = \sqrt{(1-p)/p}$; and k satisfies $k = \Phi^{-1}(\frac{b}{b+h})$ with $\Phi(\cdot)$ standing for the cumulative distribution function of the standard normal distribution. The first term in S_L represents the expected demand during the leadtime and review period, and the second term represents the safety stock against demand and yield risk in our context. With historical data only, we can first estimate the sample mean and standard deviation of D and then S_L using the above method. In the sequel, we adopt this approximation method for S_L . Under the LIR policy, orders are only placed when the current inventory position falls below such a target inventory level. We use different superscripts on q_t and π_t to distinguish among different policies.

Remark 1. It should be noted that the LIR was designed to minimize the long-run average costs, whereas, in our problem setting, the objective is to minimize the discounted costs. As a well-known and well-performed heuristic, we still use it as a benchmark under the discounted cost criterion.

Weighted newsvendor heuristic (WNH). Instead of using a single deterministic inflation factor, Chen et al. (2018) consider the possible realizations of pipeline orders and evaluate their impact on order quantities to develop a heuristic policy. Take $L = 3$ as example, which is illustrated in Fig. 1: the yield realizations of (q_{t-1}, q_{t-2}) form 4 sample paths. The terms on the right in Fig. 1 account for the cases in which (1) both outstanding orders pass the sample test, i.e., $y_1 = 1$ and $y_2 = 1$; (2) and (3) respectively, only one outstanding order passes with $(y_1 = 1, y_2 = 0)$ and $(y_1 = 0, y_2 = 1)$; and (4) both outstanding orders fail, i.e., $y_1 = 0$ and $y_2 = 0$.

The WNH policy adopts the same target inventory level S_L as in the LIR heuristic. Then, at the beginning of period $t+3$, as compared with S_L , an inventory shortfall may occur from a yield sample path. This shortfall represents an “ideal” order quantity for this sample path. If an excess inventory occurs as compared with S_L , then no order should be issued. In Fig. 1, for the sample path (2) with $y_1 = 1$ and $y_2 = 0$, the ideal order quantity should be $[S_L - I_t - q_{t-2}]^+$. To obtain the order quantity at period t , a weighted average of the ideal order quantities across different sample paths is taken, where each weight corresponds to the probability of the corresponding sample path of the yields. This is the rationale of the weighted newsvendor heuristic. Take Fig. 1 again: we have $q_t^{\text{WNH}} = p^2 [S_L - I_t - q_{t-1} - q_{t-2}]^+ + p(1-p) [S_L - I_t - q_{t-2}]^+ + [S_L - I_t - q_{t-1}]^+ + (1-p)^2 [S_L - I_t]^+$.

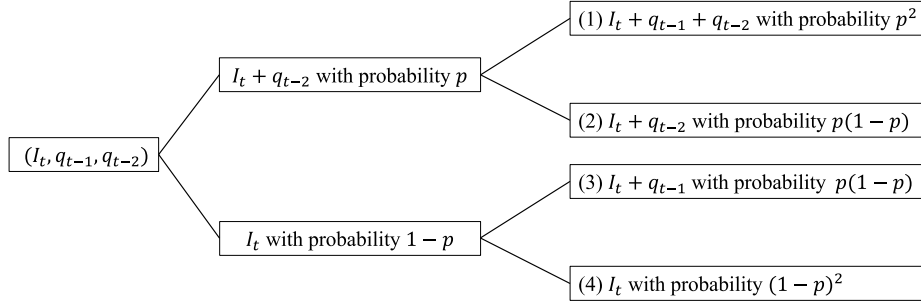


Fig. 1. Possible realizations of the pipeline orders for $L = 3$ given state $s_t = (I_t, q_{t-1}, q_{t-2})$.

In general, first, enumerate all possible outcomes of the yields for the pipeline orders $\mathbf{Q}_t$ to form 2^{L-1} sample paths, and denote $\mathcal{O} = \{(y_1, \dots, y_{L-1}) : y_i \in \{0, 1\}, 1 \leq i \leq L-1\}$ as the set of all sample paths. Second, for each sample path $o \in \mathcal{O}$, an ideal order quantity is calculated by $[S_L - I_t - \sum_{i=1}^{L-1} (y_i q_{t+i-L})]^+$. Third, the order quantity for the WNH heuristic is determined as a probability-weighted average of these ideal order quantities. That is, by including the target inventory level S_L and redefining state $s_t = (S_L, I_t, \mathbf{Q}_t)$, the order quantity for the WNH policy can be described as follows:

$$q_t^{\text{WNH}} = \pi^{\text{WNH}}(s_t) = \sum_{o \in \mathcal{O}} \left\{ p^{\sum_{i=1}^{L-1} y_i} (1-p)^{L-1-\sum_{i=1}^{L-1} y_i} \left[S_L - I_t - \sum_{i=1}^{L-1} (y_i q_{t+i-L}) \right]^+ \right\}, \quad (4)$$

where $y_i = 1$ if the order shipment q_{t+i-L} in the pipeline orders $\mathbf{Q}_t$ passes the sample test; otherwise, $y_i = 0$ for $i = 1, 2, \dots, L-1$. Since all the policies considered in the main body are time-independent (see Remark 4 for the behind reason), we omit the time index t in all functions $\pi_t(\cdot)$ for $t \in \{1, \dots, T\}$ in Eq. (4) and the following sections. In (4), the term in square brackets represents the ideal order quantity for a sample path of the pipeline orders. To be complete, let $q_t^{\text{WNH}} = [S_L - I_t]^+$, for both cases $L = 1$ and $L = 0$. Note that although the weights in Eq. (4) are constants and q_t^{WNH} is time-independent, q_t^{WNH} is state-dependent that depends on the target inventory level S_L , inventory level I_t and pipeline orders $\mathbf{Q}_t$.

Remark 2. It has been shown that the above heuristics/approximations performed well in numerical experiments. However, as mentioned in the Introduction, both have certain drawbacks. Here, we would like to add that WNH gives the “same weight” to I_t and $\tilde{I}_t$, where $I_t > \tilde{I}_t > 0$; the same can be said to each of the pipeline orders.

Below, besides to be used for benchmarks as they are the commonly used methods for the problems here, each can also serve as a “starting” solution for the DRL algorithm (to be articulated below).

4.3. Gradient descent search heuristic(GDSH)

As we mentioned in Section 1, WNH has two-fold weaknesses: (1) It is a random-yield version of the myopic solution, which tries to minimize the total expected cost for the period $L+1$; (2) The optimal policy structure for our problem is likely to be complex and remains unknown. Although WNH provides a simple functional form for calculating order quantities, it does not consider how weights depend on the current inventory level and pipeline orders. To overcome the first weakness, we propose a new heuristic policy, which adopts the gradient descent algorithm to search for the optimal weights to minimize the total expected discounted cost over the planning horizon. We call this heuristic the Gradient Descent Search Heuristic(GDSH). Specifically,

given the current state $s_t = (S_L, I_t, \mathbf{Q}_t)$, the order quantity for GDSH is formulated as:

$$q_t^{\text{GDSH}} = \pi^{\text{GDSH}}(s_t) = \sum_{o \in \mathcal{O}} \left\{ w^j \left[S_L - I_t - \sum_{i=1}^{L-1} (y_i q_{t+i-L}) \right]^+ \right\}, \quad j = 1 + \sum_{i=1}^{L-1} 2^{i-1} y_i, \quad (5)$$

where the weight w^j is fixed for all the states and time t , which can be combined into the 2^{L-1} -dimensional weight vector $\mathbf{w} = (w^1, \dots, w^{2^{L-1}})$. WNH can also be regarded as a special case of the formula (5), by setting each weight equal to the probability of the corresponding sample path of the yields, i.e., $\mathbf{w}^{\text{WNH}} \triangleq ((1-p)^{L-1}, p(1-p)^{L-2}, p^2(1-p)^{L-3}, \dots, p^{L-2}(1-p), p^{L-1})$.

Since the weight vector $\mathbf{w}$ is time-independent and state-independent, we can obtain the optimal solution $\mathbf{w}^*$ that minimizes the total expected discounted cost over the planning horizon, i.e., $\mathbf{w}^* = \arg\min_{\mathbf{w}} F_1(I_1, \mathbf{Q}_1, q_1^{\text{GDSH}})$. Suppose the value of s_t is given; we have the following result about GDSH:

Proposition 1. For q_t^{GDSH} as expressed in the formulation (5), $F_t(I_t, \mathbf{Q}_t, q_t^{\text{GDSH}})$ is convex in $\mathbf{w}$ for all t .

Proof of Proposition 1. From Proposition 1 of Chen et al. (2018), we can know that $F_t(I_t, \mathbf{Q}_t, q_t^{\text{GDSH}})$ is convex in q_t^{GDSH} for all t . Given that the composition of a convex function and an affine function is still a convex function (Boyd and Vandenberghe, 2004), $F_t(I_t, \mathbf{Q}_t, q_t^{\text{GDSH}})$ is convex in $\mathbf{w}$ for all t . $\square$

This proposition ensures that the gradient descent algorithm can find the optimal weights $\mathbf{w}^*$. In practice, the firm does not know the true distribution of demand and yield but has access to historical demand and yield observations. With these sample observations, we can substitute the true expectation with a sample average and use standard optimization methods to get the estimated weight vector $\hat{\mathbf{w}}$, where we use the $\hat{\cdot}$ notation to emphasize quantities estimated from data. Thus, GDSH is a data-driven method.

Remark 3. At a glance, the weight vector $\mathbf{w}$ looks time-independent, but it is optimized by considering all discounted costs over the planning horizon, whereas WNH is totally time-independent. Compared with WNH, where no optimization is involved, GDSH results from minimizing T-period costs.

4.4. The DRL approach

As we mentioned in Section 4.1, the optimal order quantity q_t^* for each period t depends on the inventory level I_t and the vector of pipeline orders $\mathbf{Q}_t$. However, the weight $\mathbf{w}$ of GDSH is state-independent and cannot capture the functional dependence on the

current state. To address this limitation, we design a multilayer feed-forward neural network model to approximate the weight vector as a function of the current state. Since the approximated weight function introduced later is state-dependent, we propose a new DRL algorithm that utilizes this tailored DNN model as a policy function approximator. This subsection first describes the policy structure, then develops a tailored DNN model to learn a well-performing ordering policy, and finally designs a new DRL algorithm aiming at minimizing the expected total discounted cost. The training details for this DRL approach are all discussed in this subsection.

Rather than outputting order quantities directly from a neural network model, we further modify the structure of GDSH as follows:

$$q_t^{\text{DRL}} = \pi^{\text{DRL}}(s_t) = \sum_{o \in \mathcal{O}} \left\{ w_t^j \left[S_L - I_t - \sum_{i=1}^{L-1} (y_i q_{t+i-L}) \right]^+ \right\}, \quad j = 1 + \sum_{i=1}^{L-1} 2^{i-1} y_i, \quad (6)$$

where the weight w_t^j changes over time and can be combined into the 2^{L-1} -dimensional weight vector $\mathbf{w}_t = (w_t^1, \dots, w_t^{2^{L-1}})$. We can reformulate the policy structure in (6) as $q_t^{\text{DRL}} = (s_t A)^+ \mathbf{w}_t$, where $A = [1, -1, -B]^T$ with a matrix $B \in \mathbb{R}^{2^{L-1} \times (L-1)}$, wherein each row vector exhausts every possible case of yield outcomes. This policy structure contracts with GDSH, where the weights do not vary with time. We can also know that GDSH is a special case of (6) by letting all $\mathbf{w}_t = \mathbf{w}$ for all t . Similarly, suppose the value of s_t is given, we have the following result:

Corollary 1. For q_t^{DRL} as expressed in the formulation (6), $F_t(I_t, \mathbf{Q}_t, q_t^{\text{DRL}}) = F_t(I_t, \mathbf{Q}_t, (s_t A)^+ \mathbf{w}_t)$ is convex in $\mathbf{w}_t$ for all t .

As F_t is convex with respect to $\mathbf{w}_t$, we can utilize this convex property to find the optimal weight $\mathbf{w}_t^*$ at each period t to minimize $F_t(I_t, \mathbf{Q}_t, q_t^{\text{DRL}})$. As to be seen, this convexity result will ensure the gradient descent search to obtain the “best weights”. To further capture the functional dependence of the weights on the current inventory level and pipeline orders, we design a multilayer feedforward neural network model to approximate the weight vector $\mathbf{w}_t$ as a function of the current state: $\hat{\mathbf{w}}_t = \zeta_\theta(s_t)$, where $\zeta_\theta(s_t)$ is the output from this network with parameter θ which takes s_t as an input. Note that the approximated weight vector $\hat{\mathbf{w}}_t$ is time-independent but state-dependent; hence, an enumeration of the state space is required before employing standard dynamic programming optimization methods or classical optimization algorithms. Therefore, the classical optimization approaches would be computationally involved because the state space can quickly get “exploded”. Also as mentioned in Section 1, DRL approaches have been proven to be effective in overcoming the curse of dimensionality. Specifically, our DRL method operates in a data-driven manner: it proceeds forward along sample paths and leverages simulated data during training, thereby avoiding the need for exhaustive state space enumeration. That is why we cannot directly obtain the weight vector through standard dynamic programming optimization methods or classical optimization methods instead of proposing a new DRL method.

We can reformulate the policy structure in the formulation (6) with the approximated weight function as follows:

$$q_t^{\text{DRL}} = (s_t A)^+ \zeta_\theta(s_t), \quad (7)$$

We try to learn the optimal weight vector $\mathbf{w}_t^*$ given the current state by iteratively updating and optimizing the parameters θ of the neural network.

Remark 4. Our neural network model in the main body does not include the time state t as an input, so the ordering policy is not explicitly dependent on time. However, the proposed DRL approach is optimized by minimizing the total expected discounted costs over the planning horizon, like GDSH. We also compare the DRL(t) algorithm

with the time index t included as an input to the network model and report its results in Appendix C. The findings indicate that DRL(t) does not show significant advantages over the original network model.

Note that none of the abovementioned methods consider the impact of time t on the order quantity. Therefore, they can be easily adjusted for the infinite-horizon scenario by modifying the objective function to minimize the expected long-run average cost without discounting.

Neural network architecture. We propose a tailored DNN model to output the approximated weight vector $\hat{\mathbf{w}}_t$. This model can be specified by parameter $\theta \triangleq (W_1, b_1, W_2, b_2, \dots, W_n, b_n)$, where n is the number of hidden layers or the depth of this network model, and W_k as well as b_k respectively represent the coefficient matrices and biases/intercept terms that connect the previous layer $k-1$ to the current layer k for all $k = 1, 2, \dots, n$. Note that in some studies, W_k is often referred to as “weights”. For ease of distinction, the present paper adopts the terms “weights” and “parameters” to refer to $\mathbf{w}_t$ in the heuristic policy and the coefficients in connected neural networks, respectively. The general architecture of the network model is illustrated in Fig. 2, which includes the following three modules:

(1) We first build a fully connected neural network by defining

$$\psi_1(s_t) = W_n \sigma(\dots \sigma(W_2 \sigma(W_1 s_t + b_1) + b_2) + \dots) + b_n, \quad (8)$$

where $\sigma(\cdot)$ denotes the vectorized activation function that applies the activation function elementwise. Specifically, our model takes the state vector s_t , which includes information about the current inventory level, and the vector of order pipelines as input. In each layer k , the input vector from the previous layer $k-1$ goes through an affine transformation first and then passes through a nonlinear activation function $\sigma(\cdot)$. We apply the function $\psi_1(\cdot)$ on the state vector to produce a latent weight matrix of size $(2^{L-1} \times 2)$, shown as a yellow square in Fig. 2. This matrix is used to impose interval constraints on each weight component $\hat{w}_t^j$ in subsequent operations.

(2) A Softmax function $\psi_2(\cdot)$ is applied row-wise to convert the latent weight matrix into a normalized matrix X , with each row $X_j = (\eta_j, 1 - \eta_j)$ and $\eta_j \geq 0$ for $j = 1, \dots, 2^{L-1}$.

(3) To reduce the search range of this algorithm and ensure better convergence, we restrict the weight range based on a teacher’s policy. In our context, we choose WNH as the teacher policy due to better performance compared with LIR policy (see Section 5 for the cost comparison). The approximated weight vector $\hat{\mathbf{w}}_t = (\hat{w}_t^1, \dots, \hat{w}_t^{2^{L-1}})$ is constrained to the set of $\mathcal{U} = \{(\hat{w}_t^1, \dots, \hat{w}_t^{2^{L-1}}) : w_j^{\text{WNH}} - \delta \leq \hat{w}_t^j \leq w_j^{\text{WNH}} + \delta, j = 1, \dots, 2^{L-1}, t = 1, \dots, T\}$, where w_j^{WNH} is the j th element of weight vector $\mathbf{w}^{\text{WNH}}$ obtained from WNH policy, and δ is a hyperparameter to control the degree of action exploration. To this end, we apply the following mapping function on the normalized matrix X :

$$\psi_3(X) = (X \odot U) \cdot [1, 1]^T, \quad (9)$$

where $U = [\mathbf{w}^{\text{WNH}} - \delta, \mathbf{w}^{\text{WNH}} + \delta]$, and $\odot$ represents the Hadamard product. The mapping function $\psi_3(\cdot)$ ensures that each weight $\hat{w}_t^j$ remains strictly within the interval $[w_j^{\text{WNH}} - \delta, w_j^{\text{WNH}} + \delta]$, achieved through the convex combination $\hat{w}_t^j = \eta_j(w_j^{\text{WNH}} - \delta) + (1 - \eta_j)(w_j^{\text{WNH}} + \delta)$. This architecture compresses the search space of the policy model from $\mathbb{R}^{2^{L-1}}$ to a compact δ -hypercube centered at $\mathbf{w}^{\text{WNH}}$, thus making the training behavior more stable and speeding up the learning process of our DRL algorithm. Adopting this mapping function is suitable for our inventory problem, but it may not be necessary for other issues.

In summary, given an input state s_t , a parameter θ , and the weight constraint range $U = [\mathbf{w}^{\text{WNH}} - \delta, \mathbf{w}^{\text{WNH}} + \delta]$ obtained from WNH policy, we can calculate the corresponding policy $\zeta_\theta(s_t)$ through the following transformations:

$$\zeta_\theta(s_t) = \psi_3 \circ \psi_2 \circ \psi_1(s_t), \quad (10)$$

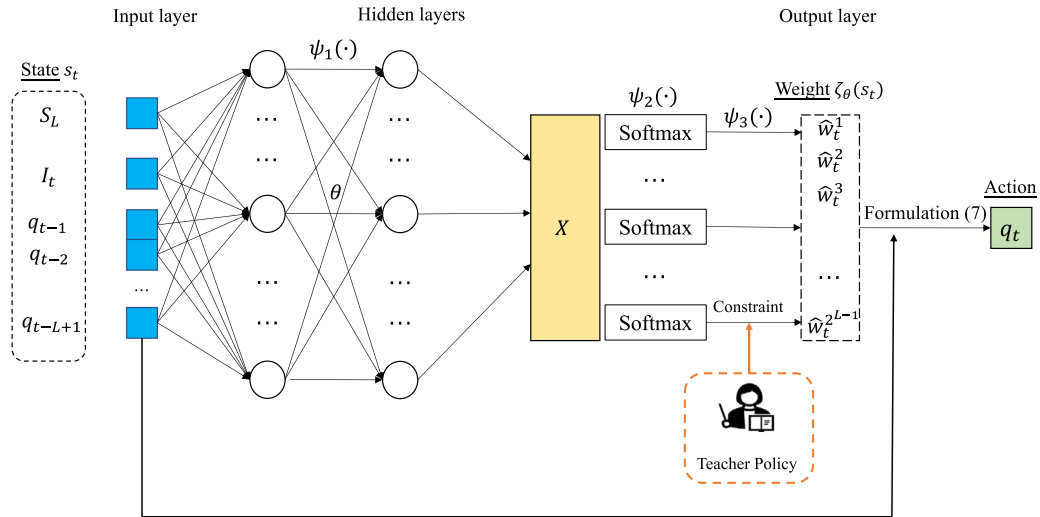


Fig. 2. Illustration of the proposed DRL agent, consisting of the tailored policy network and the new heuristic policy.

where $\circ$ denotes the composition of two functions. The proposed neural network model provides a flexible functional form by using the composition of a series of simple linear and nonlinear functions to approximate the weight vector $\mathbf{w}_t$. Then we can derive the order quantity q_t from the output of the policy network according to (7).

Blending teacher policy. DRL is a general technique for sequential decision-making problems and provides well-behaved solutions for many challenging inventory control problems (Oroojlooyjadid et al., 2022; Gijbrecchts et al., 2022). However, the process of integrating it with the results of existing well-performing exact/heuristic approaches has not yet been fully studied despite the fact that leveraging these results may improve DRL performance (Boute et al., 2022).

We use WNH as a teacher policy to steer our DRL algorithm toward the right direction. Specifically, for updating the network model (see the training algorithm part below), a loss function is defined as: $\mathcal{L} = \exp((C_T^{\text{DRL}} - C_T^{\text{WNH}})/C_T^{\text{WNH}} \times 100)$, where the total cumulative discounted cost $C_T^{\text{DRL}} = \sum_{t=1}^T \alpha^{t-1} c_t(I_t, q_{t-L}^{\text{DRL}})$ over the whole ordering horizon following the policy network is compared with the one $C_T^{\text{WNH}} = \sum_{t=1}^T \alpha^{t-1} c_t(I_t, q_{t-L}^{\text{WNH}})$ obtained from the teacher policy WNH. The benefits of adopting this loss function are two-fold. On the one hand, by comparing the performance of the policy network to a benchmark, the loss function provides a clear and intuitive measure of how well the policy network has played relative to an established baseline (Oroojlooyjadid et al., 2022). On the other hand, the original cost is an absolute value and fluctuates greatly due to uncertainties from both the demand and production sides. If the absolute cost is used as the loss function, it would likely cause the computed gradient to fluctuate greatly when using the gradient descent algorithm to update the network's parameters. This makes it difficult for the DRL algorithm to converge to the optimal solution.

Notably, our approach is different from the works of De Moor et al. (2022) and Oroojlooyjadid et al. (2022). De Moor et al. (2022) provide additional rewards at each time step for guiding the DQN agent using the potential-based shaped reward function proposed by Wiewiora et al. (2003). Oroojlooyjadid et al. (2022) use reward shaping via a feedback scheme in a multiagent and cooperative beer game, whose setting is not the focus of this paper.

Training the DRL algorithm. In the DRL context, the “DRL agent” is the one making the ordering decision, consisting of the tailored network model and the heuristic policy, as shown in Fig. 2. To ease of exposition, we omit the “DRL” superscript on the order quantities q_t^{DRL} and the total discounted cumulative cost $C_t^{\text{DRL}} = \sum_{t'=1}^t \alpha^{t'-1} c_{t'}(I_{t'}, q_{t'-L}^{\text{DRL}})$ from the initial period to period t for $t \in \{1, \dots, T\}$ in Fig. 2. The

“environment” refers to all the factors that affect the expected total discounted cumulative cost and are beyond the firm/agent's control, including demand and random yield in our problem. An “episode” represents the whole interaction process between the agent and the environment until the final time step T is reached and consists of T periods (Sutton and Barto, 2018).

The proposed DRL algorithm adopts the network model combined with the new heuristic policy mentioned in the formula (7) as a policy approximator. It updates the parameters of the network based on collected data by interacting with the environment. The goal of the agent is to find the optimal parameter θ^* of the policy network to minimize the expected total discounted cumulative cost. Fig. 3 describes the interaction process between our DRL agent and the environment in an episode. At each period $t \in \{1, 2, \dots, T\}$, the agent observes the current state s_t after including the good shipment q_{t-L} , chooses an action q_t , satisfies the realized demand d_t , and then receives a cost signal. For each episode, the interaction process consists of the following three steps:

(1) $s_t \rightarrow \zeta_\theta(\cdot) \rightarrow \hat{\mathbf{w}}_t$ is a multilayer fully connected network to output the approximated ordering weight vector $\hat{\mathbf{w}}_t$ given the current state s_t as an input. Note that the parameters of the neural network θ here are time-homogeneous, thus reducing the number of training parameters and preventing the neural network from overfitting. Then, according to (7), the agent places an order q_t based on the output from the policy network.

(2) Then the environment reacts to the action taken by the DRL agent, calculates the cost function in the current period $c_t(I_t, q_{t-L})$, and stores the cumulative cost $C_t = C_{t-1} + \alpha^{t-1} c_t(I_t, q_{t-L})$ through the $(s_{t-1}, C_{t-1}) \rightarrow C_t$ connection. The cost function $c_t(I_t, q_{t-L})$ in period t is affected by the orders q_{t-L} placed before L periods. Given that the state $s_{t-1} = (I_{t-1}, q_{t-2}, q_{t-3}, \dots, q_{t-L})$ in period $t-1$ contains information about q_{t-L} , we can transform this cost function into $c_t(I_t, q_{t-L}) = c_o s_{t-1}^T y_t \mathbf{e}_3 + c_h (s_{t-1}^T (\mathbf{e}_2 + y_t \mathbf{e}_3) - d_{t-1})$ with the unit vector $\mathbf{e}_j = [0, \dots, 1, \dots, 0]^T \in \mathbb{R}^{L+1}$ where the j th component is 1 and all other components are 0. For illustration, we use the link from s_{t-1} to C_t to capture this effect. Note that in our DRL algorithm, we only need to use the total cost of the whole episode instead of each reward signal for each period, as in the DQN algorithm (Mnih et al., 2015). The environment also leads to a new state s_{t+1} through the $(s_t, q_t) \rightarrow s_{t+1}$ connection.

(3) Once an episode is finished, the agent calculates the loss $\mathcal{L}$ based on the total discounted cumulative cost C_T , performs a gradient descent step, and updates the network parameters.

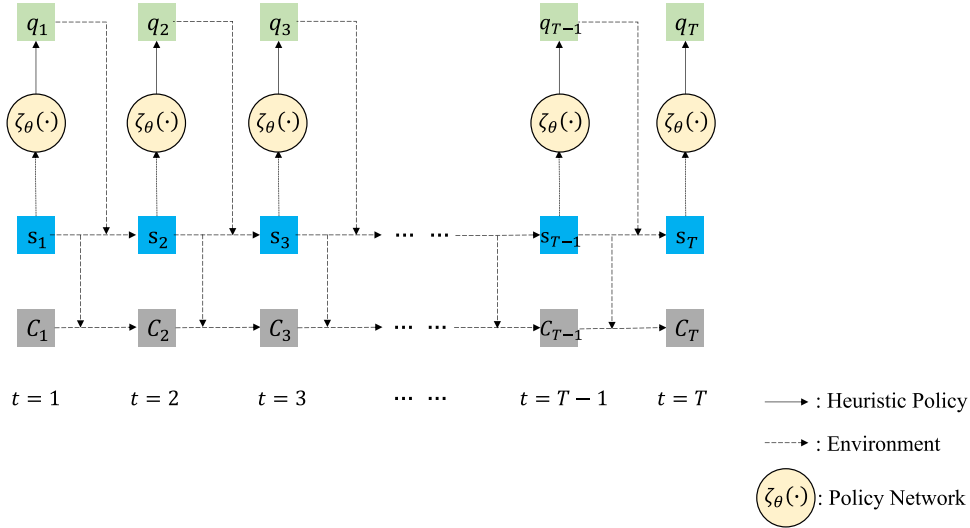


Fig. 3. Visualization of the interaction process between the proposed DRL agent and the environment.

In summary, Algorithm 1 illustrates the main idea of our algorithm. In each training iteration, training data are collected by generating samples when following the current policy $\zeta_\theta(\cdot)$ and starting from zero inventory level. Each sample contains data trajectories $\langle s_1, q_1, \dots, s_T, q_T, C_T \rangle$. To update the parameters θ , we use the batch gradient descent method. Specifically, we use the entire training dataset and calculate the average gradient for each training iteration. Although this slows down the speed of the training process, it enhances the stability of the training process relative to mini-batch gradient descent and stochastic gradient descent methods. The training procedure stops when the maximum number of training epoch is reached.

Algorithm 1

- 1: Initialize the policy network model $\zeta_\theta(\cdot)$ with parameters θ .
- 2: **repeat**
- 3: **for** each episode **do**
- 4: Set the initial inventory level equal to 0 and calculate S_L .
- 5: Calculate the total cost C_T^{WNH} and the weight range U based on WNH policy.
- 6: **for** $t = 1 : T$ **do**
- 7: Obtain the approximated weight vector $\hat{w}_t$ following the policy network $\zeta_\theta(\cdot)$ given state s_t .
- 8: Calculate action q_t according to (7).
- 9: Execute action q_t , observe the realized cost $c_t(I_t, q_{t-L})$ and next state s_{t+1} .
- 10: Store the cumulative cost $C_t = C_{t-1} + \alpha^{t-1} c_t(I_t, q_{t-L})$.
- 11: **end for**
- 12: **end for**
- 13: Calculate the loss function $\mathcal{L}$, perform a gradient descent step, and update the network parameter θ .
- 14: **until** reach the maximum number of training epoch.

5. Numerical experiments

We design two sets of experimental settings to perform a rigorous performance analysis of the proposed two methods described in Sections 4.3 and 4.4 in the inventory management problem with an all-or-nothing random yield. The first set of experiments considers a small-scale setup where the DP model can be solved. We compare the proposed methods with the optimal solution obtained by exact numerical DP methods and two known benchmark policies mentioned in Section 4.2. The second set of experiments compares these new

solutions to two known heuristics in large-size settings where the DP model is intractable.

Section 5.1 gives the implementation details. In Section 5.2, we test a set of numerical experiments selected from Chen et al. (2018) and show the near-optimality of our solutions. In Section 5.3, we design various numerical experiments with different demand distributions and cost values, after which we present the realized cost results under these experimental settings.

5.1. Implementation details

To train the proposed DRL algorithm, we first construct a fully connected multilayer neural network for the policy network consisting of three hidden layers. All hidden layers use the rectified linear unit (ReLU) activation function (Nair and Hinton, 2010) that introduces nonlinearity to capture complex relationships in the data. In this neural network, the number of nodes in the input layer is determined by the length of the state vector, resulting in $L + 1$ input nodes. The number of nodes in the output layer is equal to the length of the weight vector; thus, we have 2^{L-1} nodes in the output layer.

For each problem instance, we train a DRL agent according to the details described in Algorithm 1. The agent interacts with the environment to gather training data experiences, calculates the loss, and updates the network parameters using the Adam optimizer until the training stop condition is triggered. This study uses 500 episodes for training. Subsequently, the trained DRL agent with fixed parameters is then evaluated on 100 simulated random test trajectories of length T , where every test trajectory is generated with a distinct random seed without reuse. This ensures complete independence across all training and evaluation data. The two benchmark policies and GDSH are similarly evaluated on the same test simulated trajectories. In addition, we conduct a manual hyperparameter tuning on the batch size and the range hyperparameter. Specifically, we apply a batch gradient descent optimization method with an initial learning rate of 0.003 to update our network parameters. The batch size is thus equal to the size of the training dataset. The learning rate decreases exponentially with a decay factor of 0.98 after every epoch, and the minimum learning rate is 0.0001. In all the experiments with leadtime $L = 2$ and $L = 3$ (respectively, $L = 5$), the range hyperparameter δ is set to 0.25 (respectively, 0.0884). Appendix A summarizes all the hyperparameter values for our numerical experiments when training the proposed DRL method.

For a fair comparison, our proposed two methods adopt the same target inventory level S_L as the two benchmarks throughout this

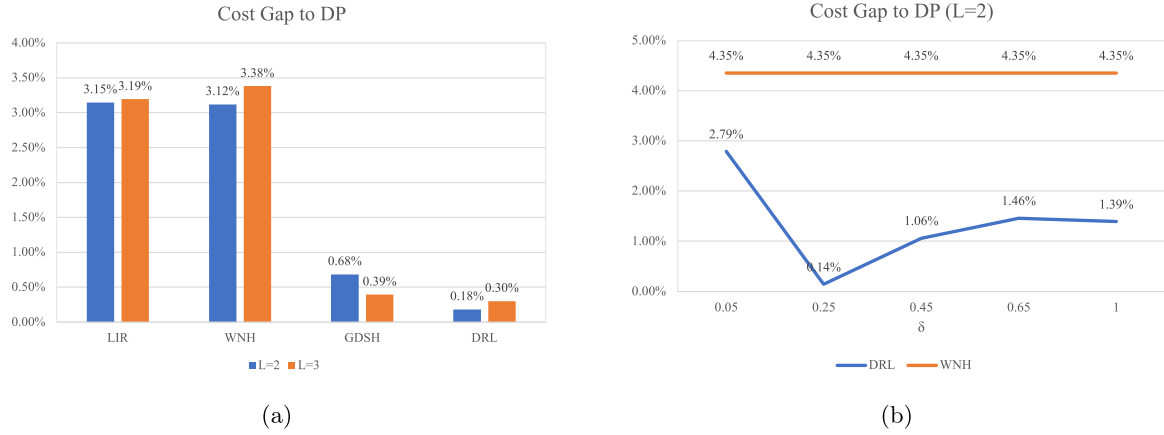


Fig. 4. Cost gap to DP on small-size inventory problems. The left subfigure shows the cost gap to DP in 972 problem instances given that $T = 10$ and $L = 2\&3$. The right subfigure shows the cost gap of our DRL method and WNH policy to DP when the range hyperparameter δ takes different values, $p = 0.7$ and $L = 2$.

numerical simulation. Therefore, any differences between these methods result from how order quantities are determined (i.e., how they adjust according to observed test results and demands). We also assume that the initial inventory is zero and that no outstanding orders arrive during the first L periods of the planning horizon for each problem instance. For a finite horizon setting, we place orders only up to the $(T - L)$ th period. Then, we can calculate the cost of periods $[T - L, T]$ according to the outlined order of events in Section 3. The same initial conditions for these four solution methods are also provided to ensure fair comparisons. As noted in Remark 1, we take the LIR as a benchmark because it is a well-established heuristic, even though it was initially proposed under the undiscounted cost criterion. We conducted all experiments on a computer with an Intel Core i5-6300HQ CPU (2.30 GHz) and 8 GB RAM, using Python 3.11.4 and PyTorch 1.13.1 for implementation.

5.2. Optimality gaps of GDSH and the proposed DRL approach

In this section, we test our approach on 972 problem instances selected from the numerical experiments of Chen et al. (2018) with the following characteristics. We consider settings with $p \in \{0.7, 0.8, 0.9\}$, unit ordering cost $c_o \in \{1.5, 2.5, 3.5\}$, time-homogeneous unit holding cost $c_h \in \{0.5, 1, 1.5\}$ and unit backlogging cost $c_b \in \{4, 5, 6\}$, when demand in each period follows a Normal distribution with mean $\mu_D \in \{6, 8, 10\}$ and standard deviation $\sigma_D \in \{1, 2\}$ given that $T = 10$ and a discount factor of 0.95. Because solving the DP model is quite computationally expensive, we only choose problem instances for leadtime $L \in \{2, 3\}$. We then calculate the optimal cost for each problem instance using the standard DP algorithm for stochastic finite horizon problems to solve the Bellman equation in which both distributions of demand and yield are priori known (Bertsekas, 2019). With randomly generated samples drawn from the same distribution of demand and yield, respectively, we train and test our new methods as described in Section 5.1.

Fig. 4 plots the results of our numerical experiment, where “Cost gap to DP” is the percentage of increased cost $(= (\text{Policy cost} - \text{DP cost}) / \text{DP cost} \times 100\%)$ and all costs are averaged over all instances. Evidently, the performances of GDSH and the proposed DRL method are close to that of the DP solution on these small-size inventory problems, while LIR and WNH increase costs by at least 3% relative to the DP solution. In other words, our new solutions achieve near-optimal performance. They also outperform the other two known heuristics by a large margin. We also test different values for the range hyperparameter δ when $L = 2$, $p = 0.7$ and compare the results with the DP solution. Fig. 4 shows that the performance of our DRL method is affected by the value of δ , but the cost gap relative to

Table 2

The average cost result of different decision-making methods over 432 problem instances. The first row reports the absolute results of different decision-making methods, and the second row reports the cost gaps (%) relative to LIR.

	LIR	WNH	DRL	GDSH
Average Cost	4101	4076	3953	3960
Cost Gap(%)	–	–0.6	–3.6	–3.44

the DP solution is still within a small range and outperforms against WNH. Our method results in only a 0.14% cost increase compared to DP when $\delta = 0.25$. Recall that when designing a neural network to output a weight vector, we constrain this weight vector based on the probability vector obtained from a teacher policy, specifically WNH policy, as mentioned earlier. Therefore, when $\delta = 1$, the constraint does not work. As observed in Fig. 4, applying this constraint based on the teacher strategy, with manual hyperparameter tuning, reduces the cost by at least 1% compared to the scenario without the teacher policy.

5.3. Sensitivity analysis

In this section, we provide a numerical sensitivity analysis of the performance of our proposed methods for larger demand support, longer time horizons, and different yield levels, where the DP solution is not achievable because of the curse of dimensionality problem. We conduct different numerical experiments where the leadtime L takes a value of 2 or 5, and the finite-horizon length T similarly takes a value of 50 or 100 with the discount factor $\alpha = 0.99$. We further investigate two cases where the demand in each period follows a normal and gamma distribution, respectively. The latter is designed to account for significant variations in demand and asymmetrical demand distributions. The mean of both demand distributions remains unchanged at $\mu_D = 10$. For the normally distributed demand, we select two different standard deviation values ($\sigma_D \in \{2, 5\}$); for the gamma distributed demand, we choose two different scale values ($\beta_D \in \{1.6, 3.2\}$). The demand sampled from these two demand distributions is truncated to zero if the value is negative. In all our experiments, we set per unit ordering cost $c_o = 2$ and per unit holding cost $c_h = 1$. We also test three different values for the unit backlogging cost under three different ratios $c_b / (c_b + c_h) \in \{0.85, 0.9, 0.95\}$. Moreover, we investigate nine values for the Bernoulli distributed yield factor ($p \in \{0.55, 0.6, 0.65, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95\}$) to allow for different sample test probabilities. Thus, we test total 432 large-size problem instances, and we will report the summary results.

We evaluate the cost performance of our DRL algorithm and GDSH compared with LIR. The average cost (gap) across all instances is given

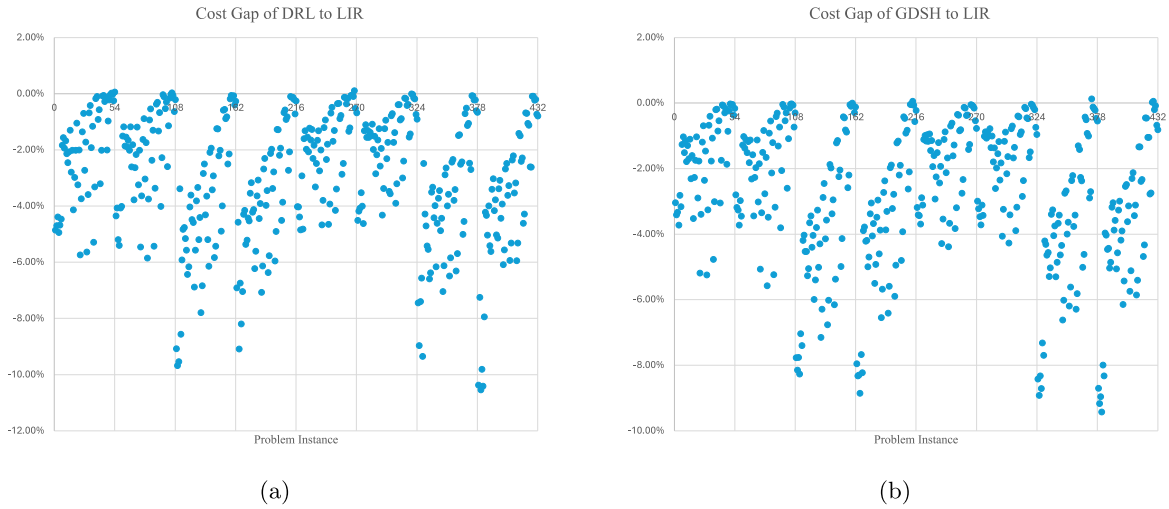


Fig. 5. Gap comparison of our DRL algorithm and GDSH with LIR policy in 432 problem instances.

Table 3

Average cost results of different decision-making methods under various parameter settings using LIR as a benchmark. We report the average gaps (%) relative to LIR method in parentheses.

Parameter	Value	Normal demand				Gamma demand			
		LIR	WNH	DRL	GDSH	LIR	WNH	DRL	GDSH
T	50	3387	3361 (-0.74)	3258 (-3.79)	3267 (-3.54)	3533	3510 (-0.63)	3405 (-3.62)	3408 (-3.52)
	100	4630	4602 (-0.60)	4466 (-3.53)	4472 (-3.41)	4855	4830 (-0.52)	4682 (-3.56)	4692 (-3.35)
L	2	2873	2871 (-0.05)	2804 (-2.40)	2814 (-2.02)	3028	3027 (-0.02)	2965 (-2.10)	2972 (-1.86)
	5	5144	5092 (-1.01)	4921 (-4.33)	4924 (-4.27)	5359	5313 (-0.87)	5122 (-4.43)	5128 (-4.31)
$c_b/(c_b + c_h)$	0.85	3032	3029 (-0.10)	2956 (-2.51)	2959 (-2.40)	3188	3187 (-0.04)	3106 (-2.57)	3110 (-2.44)
	0.9	3681	3666 (-0.42)	3562 (-3.22)	3568 (-3.06)	3862	3849 (-0.34)	3738 (-3.22)	3743 (-3.07)
	0.95	5311	5250 (-1.15)	5068 (-4.57)	5080 (-4.36)	5531	5475 (-1.03)	5286 (-4.43)	5297 (-4.24)
p	0.55	5456	5507 (0.93)	5110 (-6.33)	5114 (-6.27)	5611	5670 (1.04)	5231 (-6.78)	5242 (-6.59)
	0.6	4768	4739 (-0.61)	4582 (-3.91)	4607 (-3.38)	4901	4880 (-0.43)	4710 (-3.90)	4737 (-3.34)
	0.65	4459	4391 (-1.51)	4277 (-4.07)	4295 (-3.67)	4595	4542 (-1.14)	4431 (-3.55)	4442 (-3.33)
	0.7	4205	4128 (-1.84)	4014 (-4.55)	4026 (-4.28)	4370	4296 (-1.69)	4189 (-4.16)	4195 (-4.01)
	0.75	3929	3870 (-1.50)	3775 (-3.92)	3775 (-3.91)	4090	4034 (-1.38)	3938 (-3.71)	3942 (-3.62)
	0.8	3696	3660 (-0.97)	3569 (-3.42)	3572 (-3.35)	3885	3843 (-1.09)	3750 (-3.49)	3751 (-3.45)
	0.85	3434	3417 (-0.49)	3352 (-2.41)	3352 (-2.40)	3647	3628 (-0.54)	3554 (-2.55)	3555 (-2.52)
	0.9	3188	3184 (-0.12)	3149 (-1.23)	3150 (-1.20)	3424	3419 (-0.15)	3379 (-1.32)	3378 (-1.35)
	0.95	2938	2938 (-0.01)	2932 (-0.21)	2933 (-0.17)	3221	3221 (-0.01)	3209 (-0.38)	3210 (-0.34)
	Average	4008	3982 (-0.66)	3862 (-3.64)	3869 (-3.47)	4194	4170 (-0.56)	4043 (-3.59)	4050 (-3.43)

in Table 2, while the relative cost results for all instances are presented in Fig. 5. As depicted in Fig. 5, although GDSH performs slightly worse than LIR in a few cases, it still achieves an overall cost reduction of 3.44% compared to LIR. The results also indicate that the proposed DRL method outperforms LIR across nearly all problem instances, achieving an average cost savings of 3.6%.

Table 3 reports the average absolute cost and the cost improvement of these four policies when LIR is used as the benchmark under various problem settings. We also have the following observations:

(1) There is no significant difference in their relative cost performance of four methods under two demand distributions: normal and gamma distributions with the same mean value. Although the variance of the latter is slightly larger than that of the normal distribution, the probability of abnormal demand samples generated by gamma distributions under the existing problem settings is minimal, resulting in relatively close cost performance. Besides, as the lead time increases, the performance of binary tree-based methods (i.e., WNH, DRL, GDSH) also improves relative to LIR.

(2) As the critical ratio $c_b/(c_b + c_h)$ increases, unit backlogging cost c_b increases when unit holding cost c_h is fixed, thus leading to an unsurprising rise in the cost of all three methods. However, we can see that the costs obtained from the two benchmark methods are higher than those of our methods. As the critical ratio increases, the cost gaps between our proposed two new solutions and LIR also increase. For example, in the case of $c_b/(c_b + c_h) = 0.95$ and normally distributed demand, the gap can be as large as 4.57% (respectively, 4.36%) for our DRL approach (respectively, GDSH) compared with LIR.

(3) The cost gap of binary tree-based methods to LIR decreases as yield uncertainty decreases, i.e., p becomes bigger, especially when $p \geq 0.9$. However, with $p = 0.55$ and gamma-distributed demand, WNH performs worse than LIR; GDSH and DRL outperform LIR with greater margins (6.59% and 6.78%, respectively). Note that when p is equal to or greater than 0.75, GDSH and DRL perform almost equally.

In summary, the cost improvements from our proposed methods are notably observed under different numerical settings. Although the gain gradually diminishes with the increase in the yield probability, our method can reduce cost against the benchmark policy in most problem instances. The proposed DRL algorithm slightly outperforms GDSH but requires additional computational resources. Hence, GDSH could be a good alternative when the computational effort is of concern. Since DRL algorithms operate as black boxes, we further analyze the ordering behavior exhibited by the trained DRL agent for two representative problem instances in [Appendix B](#). Compared with the two benchmark policies, our proposed DRL algorithm can more intelligently respond to the current inventory state information and adjust order quantities by modeling non-linear representations, thus reducing costs. The model addressed so far assumes arbitrary order sizes. We extend it in [Appendix D.1](#) to discuss batch ordering. We have also assumed that demands over time are IID. This assumption can be relaxed, as shown in [Appendix D.2](#).

6. Conclusion

This paper investigates a single-item, periodic-review inventory system with stochastic demand, an all-or-nothing random yield pattern, and non-zero leadtimes in a finite horizon where the unmet demand is backlogged. The firm aims to find an optimal ordering policy to minimize the expected total discounted cost over the planning horizon, including purchase, inventory holding, and shortage costs. However, the optimal policy remains unknown, and the corresponding structure is likely to be complicated due to the challenges intrinsic to an inventory problem with an all-or-nothing random yield model.

To solve this inventory problem, this paper first draws on the structure of an existing heuristic policy and proposes a new heuristic with state-independent weights. We prove that the total cost to be computed is convex in the weight vector, which facilitates the search. Moreover, we propose a new DRL algorithm with state-dependent weights based on a tailored neural network architecture. Furthermore, this algorithm combines the results of existing heuristics to improve performance and stabilize the training process. Extensive numerical studies demonstrate that our methods can achieve near-optimal performance compared to the optimal DP solution and outperform two known benchmark policies, especially in cases with a moderate yield possibility. Given the computational burden of DRL, the new heuristic seems a well-performing method.

While the findings demonstrate that our methods can effectively address an inventory problem with an all-or-nothing yield pattern, we recognize that our methods rely on a specific form of distribution for random yield, i.e., the all-or-nothing pattern. It would be interesting to explore the application of reinforcement learning on different models for uncertain supply, such as binomial and stochastically proportional yield.

Table A.1

The hyperparameter values for numerical experiments when training the proposed DRL method.

Hyperparameter name	Value
Training episodes	500
Test episodes	100
Hidden layers	3
# nodes input layer	L
# nodes output layer	2^{L-1}
# nodes hidden layer	[8, 16, 16]
Connection	Dense
Activation function hidden layers	ReLU
Activation function output layers	Linear
Optimizer	Adam
The initial learning rate	0.003
Batch size	500
Training epoch	300

CRedit authorship contribution statement

Yuting Yan: Writing – review & editing, Writing – original draft, Methodology, Conceptualization. **Frank (Youhua) Chen:** Writing – review & editing, Validation, Supervision, Funding acquisition. **Zhe Fu:** Visualization, Validation, Methodology. **Wenjie Bi:** Validation, Supervision, Resources, Funding acquisition.

Acknowledgments

This work was supported by the National Social Science Fund of China (No. 24AJY019, No. 23BJL126); National Natural Science Foundation of China (No. W2511078); and Hong Kong RGC Project (No. C1143-20G).

Appendix A. Hyperparameters

[Table A.1](#) presents the hyperparameters of the proposed DRL algorithm used in all numerical experiments.

Appendix B. Ordering behavior analysis

Given that there is a total of 432 problem instances, we select two representative experiments to analyze the ordering strategies of heuristics, excluding cases where leadtime is equal to 5 from the visual analysis. Notably, the policy performance of each method depends on the characteristics of the specific problem instance, but the analysis patterns are similar.

[Fig. B.1](#) visualizes the ordering behaviors of different policies in two experiments where demand follows a normal distribution and a gamma distribution, respectively. [Fig. B.2](#) reports the increment of the corresponding order quantities for LIR and WNH policies compared with our DRL algorithm. We can observe that order quantities from LIR policy have a linear relationship with inventory levels and order pipelines. However, the ordering strategy of our DRL algorithm presents a more complex relationship relative to current state variables due to the introduction of activation functions by neural networks to represent nonlinear relationships. Upon observing the red area in the lower left corner of [Figs. B.1](#) and [B.2](#), it is evident that LIR and WNH policies order more products than DRL, particularly when inventory levels are below -170 and in-transit orders are below 50. This leads to a significant increase in the cost of LIR and WNH. In addition, for some states with high pipeline orders, WNH and LIR policies show a preference for fewer order quantities than DRL. This outcome may be attributed to these two benchmarks being overly optimistic about the arrival of products in transit. In contrast, there is a high probability that the latest shipment will be 0 if the sample test fails. Overall, DRL can more intelligently adjust order quantities by introducing nonlinear representations, ultimately reducing costs.

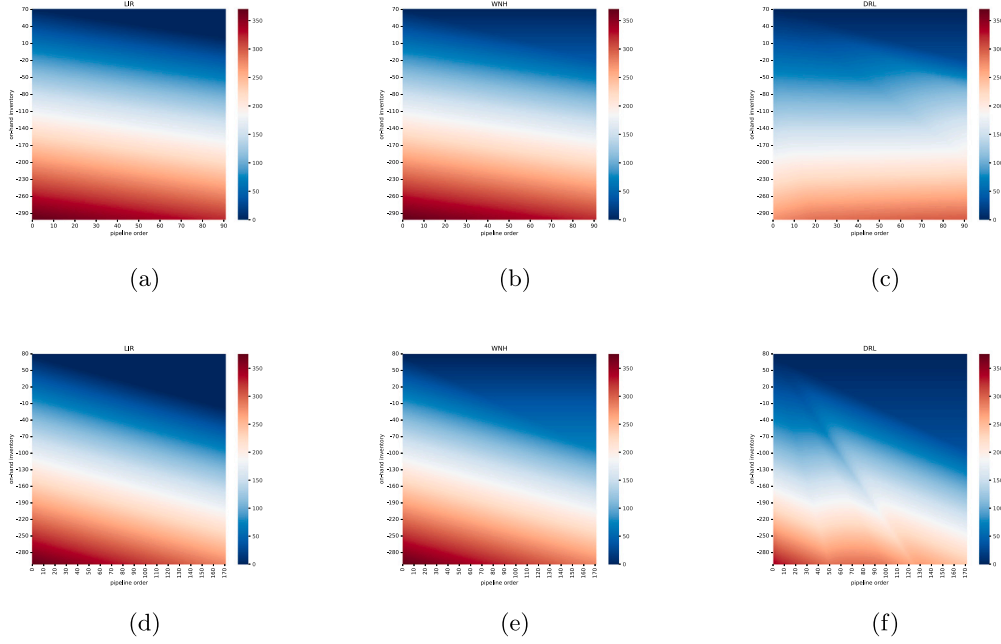


Fig. B.1. Visualization of different policies in two experiments. Images on the top (bottom) show the results of the case $\sigma_D = 2$ ($\sigma_D = \sqrt{32}$) when demand is normal (gamma) distributed and other parameters of instances are kept the same, i.e., $L = 2, T = 50, p = 0.55, c_b/(c_b + c_h) = 0.9$. For each map, the x-axis represents the vector of pipeline orders, while the y-axis represents the current inventory level. The top row of each experiment represents the policy and shows the order quantities in the heat map.

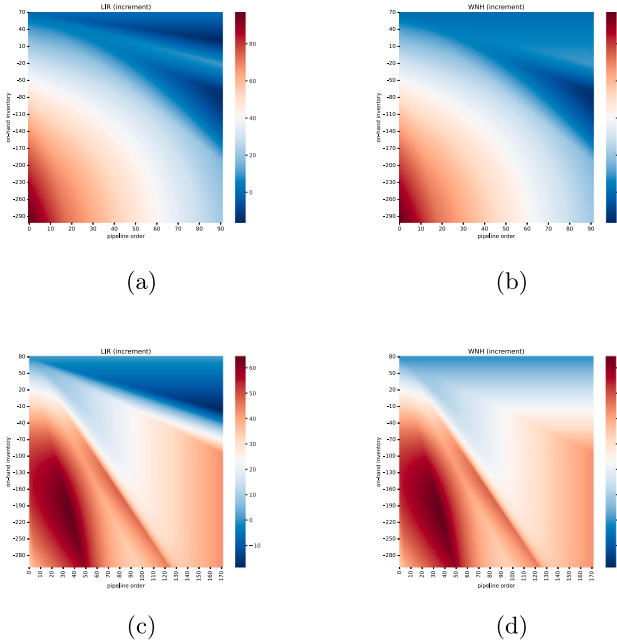


Fig. B.2. Visualization of the increment of order quantities for LIR and WNH heuristics compared with our DRL algorithm in the two experiments. Images on the top (bottom) show the results of the case $\sigma_D = 2$ ($\sigma_D = \sqrt{32}$) when demand is normal (gamma) distributed and other parameters of instances are kept the same, i.e., $L = 2, T = 50, p = 0.55, c_b/(c_b + c_h) = 0.9$.

Appendix C. The time-dependent DRL approach

The proposed DRL approach presented in the main body does not incorporate time t as an input to its neural network model, resulting in a time-independent ordering policy. However, in a finite-horizon prob-

Table C.1

The average cost result of different decision-making methods over 432 problem instances. The first row reports the absolute results of different decision-making methods, and the second row reports the cost gaps (%) relative to LIR.

	LIR	WNH	DRL	GDSH	DRL(t)
Average Cost	4101	4076	3953	3960	3952
Cost Gap(%)	—	−0.6	−3.6	−3.44	−3.6

lem, the optimal order quantities should intuitively be time-dependent. For completeness and rigor, this section provides a comparative analysis of a modified DRL algorithm that explicitly considers the influence of time t on order quantities, denoted as DRL(t). Except for including the time index t as an additional input feature, the neural network architecture and all other training details for DRL(t) remain identical to those of the proposed DRL algorithm in the main body.

Table C.1 reports the average cost of DRL(t) over 432 problem instances. The result indicates that the performance of the two DRL algorithms is highly close, with no significant advantage observed for DRL(t). One might think that the weight constraint in the neural network, which restricts the learned weight vector to a compact δ -hypercube centered at $\mathbf{w}^{\text{WNH}}$, could affect the ability of DRL(t) to express time-dependent policies. To test this hypothesis, we further conduct an ablation study on the DRL(t) variant without weight constraints. On 50 randomly sampled problem instances, the unconstrained version of DRL(t) exhibits a 1.7% higher average cost than the proposed DRL algorithm with constrained weight space. This result shows that the weight constraint is not the cause of DRL(t)'s limited improvement; instead, they are crucial for improving the algorithm's performance.

A more plausible explanation for DRL(t)'s lack of significant advantage lies in the design of the loss function, which minimizes the total cumulative discounted cost over the entire planning horizon. This formulation does not distinguish the impact of order decisions at different states and time periods on the loss function. As a result, even with time step t as an input, DRL(t) fails to utilize temporal

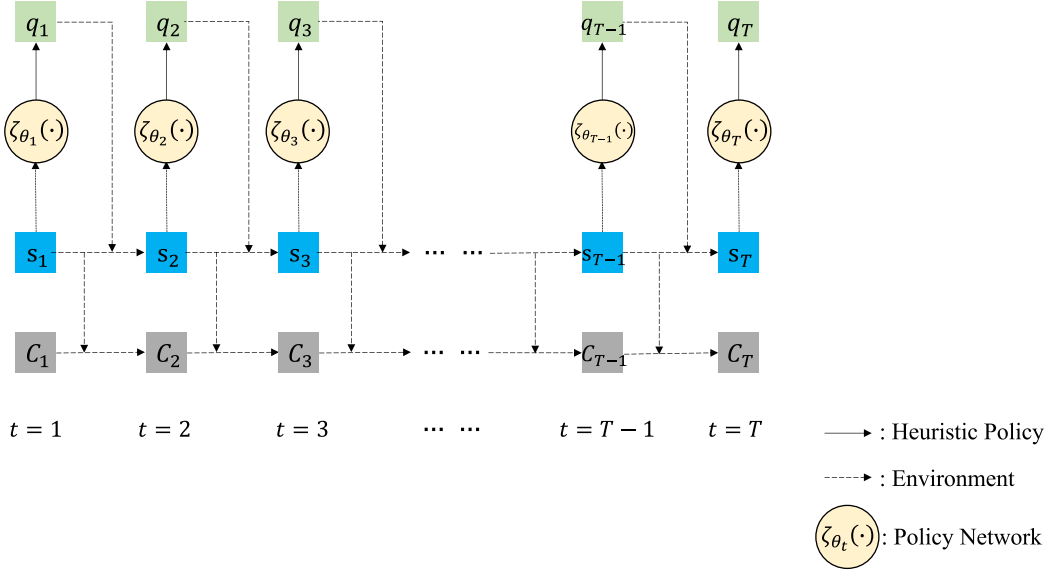


Fig. D.1. Visualization of the interaction process between the proposed DRL agent and the environment under the non-iid demand case.

information effectively and provides no additional benefit under the current optimization formulation. In contrast, although the proposed DRL algorithm appears to be time-independent in structure, it is optimized by minimizing the total cumulative discounted cost over the entire planning horizon. Moreover, this loss function is specifically designed to compare the total cumulative cost over the planning horizon of the proposed DRL approach with that of WNH. This comparative structure makes it non-trivial to modify the loss function, for instance, by replacing it with the sum of future costs affected by the current ordering decision. Future work could explore alternative loss functions that more directly consider the impact of time-dependent policies.

Appendix D. Discussions

Our current paper considers the inventory control problem with two key assumptions: ordering an arbitrary amount and IID demand. In this section, we further discuss the potential extension scenarios of our method including batch ordering and non-iid demand settings.

D.1. Batch ordering

The all-or-nothing random yield pattern is often observed in biomanufacturing or physio-chemical processes, wherein batch ordering is quite common. In this setting, at each period t , orders can only be placed in multiples of a given positive constant Q , rather than arbitrary amounts (Veinott, 1965). That is, the resulting order quantity must be an integer multiple of Q , i.e., $q_t^b = b_t Q$, where b_t represents the batch size at period t . We keep the same notations for the batch ordering setting as in Section 3, except that we use q_t^b to represent the resulting order quantity and $\mathbf{Q}_t^b$ to represent the vector of outstanding orders at period t . As we explained in Section 4.4, q_t is the original output quantity from the DRL agent at period t based on Eq. (6) and the weight vector $\mathbf{w}_t$ output from our neural network model. As well understood, the order of events is also the same as Section 3. At the beginning of each period t , the firm determines the batch size b_t based on the current inventory level I_t and the vector of outstanding orders in transit $\mathbf{Q}_t^b = (q_{t-1}^b, q_{t-2}^b, \dots, q_{t-L+1}^b) = (b_{t-1}Q, b_{t-2}Q, \dots, b_{t-L+1}Q)$. The objective is to minimize the total expected discounted cost over the planning

horizon. The dynamic programming formulation for this batch ordering setting is given by

$$\begin{aligned} V_t(I_t, \mathbf{Q}_t^b) &= \min_{b_t \in \mathbb{Z}_+} \mathbb{E}[c_o b_t Q Y_{t+L} + \alpha^L H(I_{t+L}) \\ &\quad + \alpha V_{t+1}(I_t + Y_{t+1} b_{t+1-L} Q - D_t, \mathbf{Q}_{t+1}^b)] \\ &\triangleq \min_{q_t^b \in \{0, Q, 2Q, \dots\}} \{F_t(I_t, \mathbf{Q}_t^b, q_t^b)\}, \end{aligned} \quad (\text{D.1})$$

where $I_{t+L} = I_t + \sum_{i=1}^L Y_{t+i} b_{t+i-L} Q - D_{[t,t+L-1]}$ and $D_{[t,t+L-1]} = \sum_{i=0}^{L-1} D_{t+i}$ with boundary conditions $V_{T+1} = 0$.

For the deep reinforcement learning's implementation, the DRL agent can first output a continuous order quantity q_t , which is then rounded up to the nearest multiple of Q to determine the batch size $b_t = \lceil q_t / Q \rceil$. The neural network model can learn the ordering policy by updating its parameters to minimize the loss function defined on Section 4.4.

D.2. Non-independent and identically distributed (non-iid) demands

In the non-iid demand scenario, although the demand patterns can change over time, the corresponding DP formulation remains consistent with Eq. (1). We can train our DRL agent separately for each period t , ensuring that the random demand within the same period follows an independent and identical distribution (Mao et al., 2024).

Fig. D.1 shows the interaction process between our DRL agent and the environment in an episode under the non-iid demand case. Specifically, for each period t , a separate DRL agent with parameters θ_t processes the current state s_t to determine the order quantity q_t . Each separate DRL agent has the same policy structure described in Fig. 2 with different parameters θ_t . The agent tailored at each period incurs a cost for its interactions with the environment. The total cost C_T for each episode, is then used to calculate the loss function that updates each DRL agent. This method allows the DRL agent to be specifically tailored to the demand patterns of that period. The objective is to find the optimal parameter θ_t^* of the policy network for each separate DRL agent to minimize the expected total discounted cumulative cost. Besides, we calculate the target inventory level $S_{L,t}$ for each period t based on the approach outlined in Inderfurth and Kiesmüller (2015), noting that their approach is applicable only in the iid demand case.

Table D.1

Cost results of different decision-making methods under the non-iid demand case with $L = 2$, $h = 1$, $b = 9$, $p = 0.7$ and $T = 100$. The first row reports the absolute results of different decision-making methods, and the second row reports the cost gaps (%) relative to DRL/GDSH method.

	LIR	WNH	DRL	GDSH
Average Cost	3351	3309	3134	3134
Cost Gap(%)	6.92	5.58	–	–

Consequently, it previously required taking x data batches (e.g., 500 training episodes in our numerical experiments) to train our DRL agent in a stationary demand environment with length T , but it now necessitates $x * T$ batches in a time-varying environment. While it requires more computational resources, our DRL approach can still effectively capture demands' time-varying nature.

We test the above described methods with $L = 2$, $h = 1$, $b = 9$, $p = 0.7$ and $T = 100$. In each period, the demand data is generated by summing two sampled demand distributions. The first distribution is a normal distribution with mean $\mu_D = 10$ and standard deviation $\sigma_D = 2$, and the second is also a normal distribution with a mean that can take values from the set $\{-5, -1, 0, 1, 5\}$ and a standard deviation of 1. If the sum is negative, we take it as zero. This setup cycles every five periods. Table D.1 reports the cost results. We observe that our DRL approach and GDSH achieve the lowest cost, and the LIR policy (respectively, WNH policy) increased costs by 6.92% (respectively, 5.58%) relative to our DRL approach and GDSH. Our proposed methods continue to demonstrate their superiority, even under the challenging non-iid case, outperforming the benchmark solutions.

Data availability

Data will be made available on request.

References

- Bellman, R., 1966. Dynamic programming. *Science* 153 (3731), 34–37. <http://dx.doi.org/10.1126/science.153.3731.34>.
- Bertsekas, D., 2019. *Reinforcement Learning and Optimal Control*, vol. 1, Athena Scientific (Chapter 1).
- Bollapragada, S., Morton, T.E., 1999. Myopic heuristics for the random yield problem. *Oper. Res.* 47 (5), 713–722. <http://dx.doi.org/10.1287/opre.47.5.713>.
- Boute, R.N., Gijsbrechts, J., van Jaarsveld, W., Vanvuchelen, N., 2022. Deep reinforcement learning for inventory control: A roadmap. *European J. Oper. Res.* 298 (2), 401–412. <http://dx.doi.org/10.1016/j.ejor.2021.07.016>.
- Boyd, S.P., Vandenberghe, L., 2004. *Convex Optimization*. Cambridge university press (Chapter 3).
- Bu, J., Gong, X., Yao, D., 2020. Constant-order policies for lost-sales inventory models with random supply functions: Asymptotics and heuristic. *Oper. Res.* 68 (4), 1063–1073. <https://dl.acm.org/doi/10.5555/3104322.3104425>.
- Chen, F.Y., Hu, F., Yano, C.A., Yuan, Q., 2018. Heuristics and bounds for an inventory system with an all-or-nothing yield pattern and lead-times. In: 2018 IEEE International Conference on Service Operations and Logistics, and Informatics. SOLI, Singapore, Singapore, pp. 180–184.
- De Moor, B.J., Gijsbrechts, J., Boute, R.N., 2022. Reward shaping to improve the performance of deep reinforcement learning in perishable inventory management. *European J. Oper. Res.* 301 (2), 535–545. <http://dx.doi.org/10.1016/j.ejor.2021.10.045>.
- DeCroix, G.A., 2013. Inventory management for an assembly system subject to supply disruptions. *Manag. Sci.* 59 (9), 2079–2092. <http://dx.doi.org/10.1287/mnsc.1120.1660>.
- Federgruen, A., Yang, N., 2011. Procurement strategies with unreliable suppliers. *Oper. Res.* 59 (4), 1033–1039. <http://dx.doi.org/10.1287/opre.1110.0935>.
- Gijsbrechts, J., Boute, R.N., Van Mieghem, J.A., Zhang, D.J., 2022. Can deep reinforcement learning improve inventory management? Performance on lost sales, dual-sourcing, and multi-echelon problems. *Manuf. Serv. Oper. Manag.* 24 (3), 1349–1368. <http://dx.doi.org/10.1287/msom.2021.1064>.
- Henig, M., Gerchak, Y., 1990. The structure of periodic review policies in the presence of random yield. *Oper. Res.* 38 (4), 634–643. <http://dx.doi.org/10.1287/opre.38.4.634>.
- Huh, W.T., Nagarajan, M., 2010. Linear inflation rules for the random yield problem: Analysis and computations. *Oper. Res.* 58 (1), 244–251. <http://dx.doi.org/10.1287/opre.1080.0639>.
- Inderfurth, K., 2009. How to protect against demand and yield risks in MRP systems. *Int. J. Prod. Econ.* 121 (2), 474–481. <http://dx.doi.org/10.1016/j.ijpe.2007.02.005>.
- Inderfurth, K., Kiesmüller, G.P., 2015. Exact and heuristic linear-inflation policies for an inventory model with random yield and arbitrary lead times. *European J. Oper. Res.* 245 (1), 109–120. <http://dx.doi.org/10.1016/j.ejor.2015.03.006>.
- Inderfurth, K., Vogelgesang, S., 2013. Concepts for safety stock determination under stochastic demand and different types of random production yield. *European J. Oper. Res.* 224 (2), 293–301. <http://dx.doi.org/10.1016/j.ejor.2012.07.040>.
- Kaynov, I., van Knippenberg, M., Menkovski, V., van Breemen, A., van Jaarsveld, W., 2024. Deep reinforcement learning for one-warehouse multi-retailer inventory management. *Int. J. Prod. Econ.* 267, 109088. <http://dx.doi.org/10.1016/j.ijpe.2023.109088>.
- Kouvelis, P., Xiao, G., Yang, N., 2018. On the properties of yield distributions in random yield problems: Conditions, class of distributions and relevant applications. *Prod. Oper. Manag.* 27 (7), 1291–1302. <http://dx.doi.org/10.1111/poms.12869>.
- Li, X., 2017. Optimal procurement strategies from suppliers with random yield and all-or-nothing risks. *Ann. Oper. Res.* 257, 167–181. <http://dx.doi.org/10.1007/s10479-015-1923-4>.
- Li, Q., Xu, H., Zheng, S., 2008. Periodic-review inventory systems with random yield and demand: Bounds and heuristics. *IIE Trans.* 40 (4), 434–444. <http://dx.doi.org/10.1080/07408170701598108>.
- Mao, W., Zhang, K., Zhu, R., Simchi-Levi, D., Başar, T., 2024. Model-free nonstationary reinforcement learning: Near-optimal regret and applications in multiagent reinforcement learning and inventory control. *Manag. Sci.* <http://dx.doi.org/10.1287/mnsc.2022.02533>.
- Martagan, T., Akcay, A., Koek, M., Adan, I., 2020. Optimal production decisions in biopharmaceutical fill-and-finish operations. *IIE Trans.* 53 (2), 149–163. <http://dx.doi.org/10.1080/24725854.2020.1770902>.
- Martagan, T., Krishnamurthy, A., Maravelias, C.T., 2016. Optimal condition-based harvesting policies for biomufacturing operations with failure risks. *IIE Trans.* 48 (5), 440–461. <http://dx.doi.org/10.1080/0740817X.2015.1101523>.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., et al., 2015. Human-level control through deep reinforcement learning. *Nature* 518 (7540), 529–533. <http://dx.doi.org/10.1038/nature14236>.
- Nair, V., Hinton, G.E., 2010. Rectified linear units improve restricted boltzmann machines. In: *Proceedings of the 27th International Conference on Machine Learning. ICML-10, Haifa, Israel*, pp. 807–814.
- Oroojlooyjadid, A., Nazari, M., Snyder, L.V., Takáč, M., 2022. A deep Q-network for the beer game: Deep reinforcement learning for inventory optimization. *Manuf. Serv. Oper. Manag.* 24 (1), 285–304. <http://dx.doi.org/10.1287/msom.2020.0939>.
- Powell, W.B., 2007. *Approximate Dynamic Programming: Solving the Curses of Dimensionality*, vol. 703, John Wiley & Sons (Chapter 1).
- Puterman, M.L., 2014. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons (Chapter 6).
- Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., et al., 2016. Mastering the game of Go with deep neural networks and tree search. *Nature* 529 (7587), 484–489. <http://dx.doi.org/10.1038/nature16961>.
- Sutton, R.S., Barto, A.G., 2018. *Reinforcement Learning: An Introduction*. MIT press (Chapter 3).
- Tang, S.Y., Gurnani, H., Gupta, D., 2014. Managing disruptions in decentralized supply chains with endogenous supply process reliability. *Prod. Oper. Manag.* 23 (7), 1198–1211. <http://dx.doi.org/10.1111/poms.12160>.
- Tomlin, B., 2009. Impact of supply learning when suppliers are unreliable. *Manuf. Serv. Oper. Manag.* 11 (2), 192–209. <http://dx.doi.org/10.1287/msom.1070.0206>.
- Veinott, Jr., A.F., 1965. The optimal inventory policy for batch ordering. *Oper. Res.* 13 (3), 424–432. <http://dx.doi.org/10.1287/opre.13.3.424>.
- Wang, Y., Gilland, W., Tomlin, B., 2010. Mitigating supply risk: Dual sourcing or process improvement? *Manuf. Serv. Oper. Manag.* 12 (3), 489–510. <http://dx.doi.org/10.1287/msom.1090.0279>.
- Wang, Y., Minner, S., 2024. Deep reinforcement learning for demand fulfillment in online retail. *Int. J. Prod. Econ.* 269, 109133. <http://dx.doi.org/10.1016/j.ijpe.2023.109133>.
- Wiewiora, E., Cottrell, G.W., Elkan, C., 2003. Principled methods for advising reinforcement learning agents. In: *Proceedings of the 20th International Conference on Machine Learning. ICML-03, Washington, DC, USA*, pp. 792–799.
- Yano, C.A., Lee, H.L., 1995. Lot sizing with random yields: A review. *Oper. Res.* 43 (2), 311–334. <http://dx.doi.org/10.1287/opre.43.2.311>.
- Yuan, X., Bi, G., Fei, Y., Liu, L., 2021. Supply chain with random yield and financing. *Omega* 102, 102334. <http://dx.doi.org/10.1016/j.omega.2020.102334>.